

23 — Async ETL Parallelism: ThreadPoolExecutor for I/O-Bound Pipelines

If you haven't read doc 14 yet, that one covers the overall pipeline orchestration pattern (extract, transform, load) — this doc assumes that background and focuses on a different question: once you have a working sequential pipeline, how do you make it process multiple records at the same time without breaking anything?

What it is

`bilingual_etl/scripts/main_etl.py` accepts a `--workers` flag that controls how many records are processed in parallel. When `workers=1` (the default), every category or company is processed one after the other in a simple `for` loop. When `workers` is greater than 1, the code switches to a `ThreadPoolExecutor` — Python's built-in mechanism for running multiple tasks concurrently using a pool of threads.

The key insight: our pipeline is **I/O-bound**, not CPU-bound. Each record's processing time is dominated by waiting — waiting for the LLM API to respond to a translation request, waiting for the embedding API to return vectors, waiting for the database to confirm a write. The actual computation on our side (building a narrative string, computing a hash) takes microseconds. Threads are ideal for this because while one thread is waiting on an API response, Python can switch to another thread that's ready to do work.

Real-life analogy

Imagine you're at a post office sending 500 packages. Each package needs three steps: fill out the customs form (fast, 30 seconds), hand it to the clerk (then wait 5 minutes while they weigh it, scan it, and print a label), and pay at the register (wait 2 minutes in line).

Sequential processing (workers=1): You fill out one form, hand it to one clerk, wait 5 minutes, pay, then start on the next package. You spend 90% of your time just standing around waiting.

Parallel processing (workers=4): You hire three friends. While you're waiting for your clerk to finish scanning, your friend is already handing their package to a different clerk, and another friend is filling out their customs form. Four packages are in-flight at different stages simultaneously. You finish much faster — not 4x faster (the clerks are still the bottleneck), but dramatically faster than doing everything one-at-a-time.

Why friends (threads), not clones (processes)? If each step required heavy mental math (CPU-intensive), you'd need actual separate brains (processes) because one brain can only do one calculation at a time. But since 90% of the work is waiting for someone else (the clerk, the API), a single brain can juggle multiple packages just by switching attention to whichever one is ready — that's what threads do.

Why we used it here

This ETL pipeline processes roughly 3,000 categories and 471 companies, and each record needs multiple external API calls:

- **Translation** (LLM call via `generate_with_retry`) — waiting on Gemini/Groq/OpenRouter to respond.
- **AI enrichment** (another LLM call) — more waiting.
- **Embedding generation** (embedding API call) — yet more waiting.

- **Database writes** (PostgreSQL round-trip) — waiting for commit confirmation.

For a single company, the wall-clock time might be 10-30 seconds, almost entirely API wait time. Processing 471 companies sequentially means 1.3-4 hours of mostly idle waiting.

With `--workers 4`, four companies are in-flight at once. While company A is waiting for its translation response, companies B, C, and D are at different stages of their own processing. Throughput improves significantly without needing proportionally more resources.

The worker count is deliberately a command-line flag (not hardcoded) because the optimal number depends on the API provider's rate limits — running 20 workers against a free-tier Gemini endpoint would just trigger rate-limit errors (429s) on 16 of them simultaneously, making things slower not faster due to retry backoff. The default of 1 (sequential) is safe for any provider; you dial it up based on your current provider's limits.

Code walkthrough

From `bilingual_etl/scripts/main_etl.py`:

The CLI flag:

```
parser.add_argument(  
    "--workers",  
    type=int,  
    default=1,  
    help="Number of parallel workers for processing (default: 1, sequential).",  
)
```

The default is 1 — sequential mode — so running the ETL without the flag behaves exactly as it always did. No surprise parallelism. You opt in by passing `--workers 4` (or whatever number makes sense for your provider's rate limits).

The branching logic (shown for categories; companies use the same pattern):

```
def process_categories(
    conn,
    categories: list[dict],
    masker: ProtectedTermsMasker,
    force: bool = False,
    limit: int | None = None,
    workers: int = 1,
) -> tuple[int, int, int]:
    # ...
    if workers <= 1:
        # Sequential: simple for loop
        for category in categories:
            try:
                if _process_one_category(conn, category, masker, force):
                    processed += 1
            except Exception as e:
                failed += 1
                logger.error(...)
    else:
        # Parallel: ThreadPoolExecutor
        def _worker(cat):
            worker_conn = get_connection()          # Each thread gets its own DB con
            try:
                return _process_one_category(worker_conn, cat, masker, force)
            finally:
                worker_conn.close()                 # Always close, even on failure

        with ThreadPoolExecutor(max_workers=workers) as pool:
            futures = {pool.submit(_worker, cat): cat for cat in categories}
            for future in as_completed(futures):
                cat = futures[future]
                try:
                    if future.result():
                        processed += 1
                except Exception as e:
                    skipped += 1
```

```
failed += 1
logger.error(...)
```

Walking through the parallel path piece by piece:

Each thread gets its own database connection. `_worker` calls `get_connection()` inside the thread rather than sharing the main connection. PostgreSQL connections are not thread-safe — two threads sending queries on the same connection would corrupt each other's results. This is the single most important detail in the parallel path: each thread must have its own independent connection, opened at the start and closed in a `finally` block so it's always released, even if processing that record raises an exception.

`pool.submit(_worker, cat)` hands each category to the thread pool. The pool manages a queue internally — if you have 3,000 categories and 4 workers, the pool runs 4 at a time and automatically picks up the next one whenever a worker finishes. You don't need to manually assign work to specific threads.

futures dictionary maps each Future back to its category.

When a future completes (via `as_completed()`), we look up which category it was processing so we can log a meaningful error message if it failed, not just "some unnamed future raised an exception."

`as_completed(futures)` yields futures in the order they finish, not the order they were submitted. This is important — if category #500 finishes before category #3 (because #3 hit a rate limit and had to retry), `as_completed` gives us #500 first. We don't waste time blocking on a slow item while finished items pile up unprocessed.

Error isolation: A single record's failure (an exception inside `_process_one_category`) is caught per-future and counted as `failed += 1`. It never crashes the pool or aborts the other 2,999 records. This matches the design principle in the module docstring: "a failure on one record ... must not abort a multi-hour batch run over one bad record."

Progress reporting:

```
if (processed + skipped + failed) % 5 == 0:
    _report()
```

Every 5 records (whether processed, skipped, or failed), the code writes progress to the `app_config` table. This powers the ETL Progress dashboard in the Admin UI (see doc 26). The same progress-reporting logic exists in both the sequential and parallel paths — the parallel path just has multiple records completing between reports.

Why threads, not processes

Python has a well-known limitation called the **Global Interpreter Lock (GIL)**: only one thread can execute Python bytecode at any given instant. This makes threads useless for CPU-bound work (heavy math, data crunching) because they can't truly run Python code in parallel.

But the GIL is released whenever Python calls out to an I/O operation — a network request, a file read, a database query. During that time, other threads can run their own Python code. Our pipeline spends 95% + of its time in I/O (waiting for HTTP responses from LLM/embedding APIs, waiting for PostgreSQL queries). So threads give us almost the same benefit as true parallelism here, with much less overhead:

- Threads share memory — no need to serialize/deserialize data between processes.
- Thread creation is lighter than process creation.
- No need for inter-process communication mechanisms.
- The shared `masker` object (protected terms data loaded once) is accessible to all threads without copying it.

If we were doing heavy numerical computation (like training a model), we'd need `ProcessPoolExecutor` instead. But for "wait on API, write to DB, repeat" workloads, `ThreadPoolExecutor` is the right tool.

How to explain this in an interview

"Our ETL pipeline calls external APIs — translation, enrichment, and embedding — for each of the 3,000+ records, and those API calls dominate the wall-clock time. We added optional parallelism using Python's `ThreadPoolExecutor`, which is ideal here because the workload is I/O-bound: while one thread waits for an API response, others can process their records. We chose threads over processes because the GIL is released during I/O operations, threads share memory (so the protected-terms dictionary doesn't need to be copied), and the overhead is lower. Each thread gets its own database connection to avoid thread-safety issues with PostgreSQL. The worker count is a CLI flag, not hardcoded, because the right number depends on the API provider's rate limits — too many workers just triggers rate-limit errors on all of them simultaneously. A single record's failure is caught per-future and doesn't abort the rest of the batch."